AI 智能体学习路线

面向人群

适合刚接触人工智能、会使用电脑并愿意进行基础编程实践的学习者。路线默认不要求机器学习基础，但建议具备基本的 Python 阅读能力；完全没有编程经验，也可以先完成前半段，再补充 Python。

学习目标

完成本路线后，你应当能够：

- 解释大语言模型、提示词、上下文、工具调用、工作流和智能体之间的关系。
- 使用 Python 调用模型 API，并处理结构化输出。
- 构建能够调用搜索、文件、数据库或业务 API 的简单智能体。
- 使用 RAG 让模型基于自己的文档回答问题。
- 设计多步骤任务、状态管理、人工确认和失败重试。
- 对智能体进行基本的效果、安全性、成本和延迟评估。
- 将一个小型智能体部署为可使用的应用。

总体安排

建议用 12 周完成，每周投入 6 至 10 小时。可以按自己的时间压缩或延长，但不要跳过动手项目。

阶段	时间	核心产出
0. 环境与基础	第 1 周	Python 项目与 API 调用环境
1. 大模型基础	第 2-3 周	稳定的问答与结构化输出程序
2. 工具调用与工作流	第 4-5 周	能调用外部工具的任务助手
3. RAG 知识库	第 6-7 周	基于个人文档的问答系统
4. 智能体工程化	第 8-9 周	有状态、可追踪、可评估的智能体
5. 部署与综合项目	第 10-12 周	可演示、可部署的完整应用

第 0 阶段：准备环境与编程基础

学习重点

- Python 基础：变量、函数、列表与字典、文件读写、异常处理、模块。
- 虚拟环境、依赖管理、环境变量和 Git 基础。

- HTTP、JSON、API Key、请求与响应的基本概念。
- 隐私意识：不要把真实密码、身份证号、客户数据直接发送给模型。

动手任务

1. 安装 Python 3.11 或更高版本，并创建虚拟环境。
2. 用 `.env` 文件保存 API Key，代码中通过环境变量读取。
3. 写一个程序：读取一段文本，调用模型进行摘要，并把结果保存为 Markdown 文件。
4. 为程序增加异常处理和简单日志。

阶段检查

你能够独立完成“输入文本 -> 调用模型 -> 输出结果”的闭环，并知道 API Key 不应提交到 Git 仓库。

第 1 阶段：理解大语言模型与提示词

必须理解的概念

- **Token**：模型处理文本的基本单位，影响上下文容量和成本。
- **上下文窗口**：一次请求中模型可以看到的信息范围。
- **系统提示词、用户消息、助手消息**：不同角色的指令和对话内容。
- **温度与随机性**：创造性任务可以适当提高，结构化任务通常保持较低。
- **幻觉**：模型可能生成听起来合理但不真实的内容，因此需要约束、检索和验证。
- **结构化输出**：让模型返回 JSON 或符合某个 Schema 的结果，便于程序继续处理。

提示词的基本模板

一个实用的提示词通常包含：

1. 角色：模型要扮演什么职责。
2. 目标：要完成什么任务。
3. 输入：待处理的材料和必要背景。
4. 约束：格式、范围、语气、禁止事项。
5. 示例：输入与期望输出的少量样例。
6. 校验：要求模型检查遗漏，并在信息不足时明确说明。

动手任务

- 做一个“会议纪要整理器”：输出议题、结论、待办事项、负责人和截止时间。
- 使用 JSON Schema 约束输出，并在 Python 中校验字段。

- 准备 10 条测试输入，记录成功率、错误类型和改进后的结果。

阶段检查

你不再依赖“多写几句提示词”来碰运气，而是能够把任务拆成输入、规则、输出格式和验证步骤。

第 2 阶段：工具调用与工作流

核心概念

智能体的关键不只是聊天，而是让模型根据任务选择并调用工具。工具可以是：

- 搜索或网页读取。
- 计算器和日期处理。
- 文件读写。
- 数据库查询。
- 企业内部 API。
- 发送邮件、创建任务等具有副作用的操作。

建议先理解“模型负责决策，程序负责执行”的边界。模型可以提出工具调用参数，但真正执行工具、校验权限和处理错误的工作应由程序控制。

推荐的工作流结构

```
接收请求
  -> 识别任务类型
  -> 收集必要信息
  -> 调用工具
  -> 检查工具结果
  -> 生成回答或继续下一步
  -> 记录日志并返回结果
```

动手任务：个人研究助手

实现一个研究助手，至少包含三个工具：

- 计算器。
- 本地文件搜索。
- 一个公开信息查询接口。

要求：

- 工具参数使用明确的 Schema。
- 工具执行失败时能返回可读错误。

- 涉及发送、删除、付款等操作时必须人工确认。
- 在最终回答中区分“已确认事实”和“模型推断”。

阶段检查

你能解释工具调用和普通函数调用的区别，并能避免让模型直接执行未经校验的危险操作。

第 3 阶段：RAG 与个人知识库

为什么需要 RAG

模型的预训练知识不等于你的最新资料，也不等于企业内部知识。RAG，即检索增强生成，通常包含：

```
文档导入 -> 清洗 -> 分块 -> 向量化 -> 建立索引
用户问题 -> 向量化 -> 检索相关片段 -> 交给模型生成回答
```

需要掌握的知识

- 文档解析：Markdown、PDF、网页和表格。
- 分块策略：按标题、段落或语义切分，并保留元数据。
- Embedding：把文本转换为可比较的向量表示。
- 向量数据库或向量索引。
- 召回数量、相似度、重排序。
- 引用来源与“找不到答案时拒答”。

动手任务：课程资料问答库

准备一组公开课程资料，构建问答系统。回答必须包含：

- 简明结论。
- 依据内容的引用或文件名。
- 无法找到依据时的明确说明。

准备至少 20 个问题进行测试，其中包括答案在文档中、答案不在文档中、需要综合多个片段三类问题。

常见误区

- 把整本长文档直接塞进上下文。
- 只看回答是否流畅，不检查引用是否真的支持结论。
- 只用相似度检索，忽略关键词、元数据和重排序。
- 文档更新后没有更新索引。

第 4 阶段：智能体工程化

从 Demo 到可靠应用

当智能体开始处理真实任务，需要补齐以下能力：

- **状态管理**：保存用户目标、已完成步骤、工具结果和待确认事项。
- **记忆设计**：区分当前会话、用户偏好和长期业务事实，避免无限保存。
- **可观测性**：记录请求、模型版本、工具调用、耗时、错误和成本。
- **评估体系**：用固定测试集衡量任务成功率、事实准确率、引用正确率和工具调用正确率。
- **失败处理**：超时、限流、空结果、格式错误和模型拒答都要有处理路径。
- **安全控制**：权限、输入过滤、提示词注入防护、敏感数据脱敏和人工审批。

建议的工程原则

1. 先用确定性的工作流解决问题，再在确有必要时引入自主规划。
2. 把高风险动作放在模型之外，由代码执行权限和校验。
3. 所有外部数据都视为不可信输入，包括网页内容和检索到的文档。
4. 为每个工具设置超时、参数校验和最小权限。
5. 让每次模型输出都可以被测试、追踪和复现。

动手任务：客服工单分流智能体

输入一封客户邮件，完成：

- 判断问题类别和紧急程度。
- 提取订单号、产品和问题描述。
- 检索知识库中的处理方案。
- 生成草稿回复。
- 高风险或低置信度情况转人工。

评估指标：字段提取准确率、分类准确率、引用正确率、人工转接召回率、平均响应时间和单次成本。

第 5 阶段：部署与综合项目

需要掌握的组件

- 一个简单的 Web API 或 Web 界面。
- 配置管理与密钥管理。
- 数据库或对象存储。

- 日志与错误监控。
- 基础容器化和部署流程。
- 速率限制、缓存、重试和成本预算。

综合项目建议

选择一个真实但低风险的场景，例如：

- 个人学习资料教练。
- 求职材料分析与修改助手。
- 产品说明书问答系统。
- 会议资料整理和待办跟踪工具。
- 内部知识库检索助手。

项目最低要求：

- 至少一个模型调用。
- 至少两个工具或一个 RAG 知识库。
- 明确的输入输出 Schema。
- 10 至 30 条自动化测试。
- 错误处理、日志和人工确认机制。
- 一份 README，写清楚架构、启动方式、限制和已知问题。

推荐技术栈

不要一开始堆叠大量框架。建议按以下顺序选择：

- 编程语言：Python。
- 模型调用：先使用模型厂商的官方 SDK。
- 数据校验：Pydantic 或 JSON Schema。
- Web 服务：FastAPI。
- RAG 起步：本地向量索引或成熟向量数据库。
- 任务编排：先用普通 Python 函数和状态机；复杂后再选择智能体框架。
- 评估与追踪：使用测试集、结构化日志和必要的观测平台。

框架会变化，基础 API、数据结构、检索原理、测试方法和安全边界更值得长期投入。

每周学习方法

每周保持一个闭环：

1. 用 1 至 2 小时理解一个核心概念。
2. 用 2 至 4 小时完成最小可运行代码。
3. 用 1 至 2 小时设计测试样例并记录失败。
4. 用 1 小时整理 README、日志和下一步改进。

学习笔记建议记录：问题、输入、模型版本、提示词版本、输出、错误原因和修复方式。不要只保存“成功案例”。

初学者常见误区

- 一开始就学习多智能体、长链路自主规划和复杂框架。
- 把模型当作数据库，要求它记住所有事实。
- 没有测试集，只凭几次对话判断效果。
- 忽略成本、延迟、权限和隐私。
- 看到模型输出格式错误时，继续用提示词堆叠解决，而不做 Schema 校验。
- 只做聊天 Demo，没有明确用户、输入、输出和成功标准。

最终能力自测清单

- [ ] 我能独立调用模型 API，并安全管理密钥。
- [ ] 我能让模型返回可校验的结构化数据。
- [ ] 我能把外部能力封装成带参数校验的工具。
- [ ] 我能设计一个可回退、可重试的工作流。
- [ ] 我能构建带来源引用的 RAG 问答系统。
- [ ] 我能识别提示词注入、越权操作和敏感信息风险。
- [ ] 我有固定测试集，并能报告效果、成本和延迟。
- [ ] 我能把项目部署为其他人可使用的服务。

一句话路线

先学会稳定调用模型，再学会让模型调用工具；接着让它基于可靠资料工作，最后补齐状态、评估、安全和部署。每个阶段都用一个小项目验证，而不是只看教程。

版本说明： 本路线以通用能力为主，具体模型、SDK 和框架会持续变化。学习时优先参考所选模型厂商的最新官方文档，并用自己的测试集验证结论。